Ответы на контрольные вопросы ПР5 (№ 43–52)

Индивидуальный проект «Сервис планирования киносъёмок» (веб-интерфейс «КиноПлан»)

Лимонов Никита Андреевич, ЭФБО-08-24

43. Какая предметная область реализована в индивидуальном проекте?

Организация киносъёмочного процесса. Фильм состоит из сцен; для каждой сцены известен нужный размер съёмочной группы. Съёмка сцены назначается на локацию (место съёмок с ограниченной вместимостью) в определённую дату. Сервис хранит фильмы, сцены, локации и график съёмок, проверяет, вмещает ли локация группу и свободна ли она на дату, не допускает двух активных съёмок на одной локации в один день. Сущности: **фильм**, **сцена**, **локация**, **съёмка**. Пользователи — режиссёры, ассистенты, продюсеры и администраторы съёмочного процесса.

44. Какие страницы реализованы?

- главная страница — описание сервиса, сводка (сколько фильмов, локаций, активных съёмок) и кнопки перехода в разделы;
- список фильмов;
- страница фильма — режиссёр, таблица сцен и съёмки этого фильма;
- список локаций — таблица, отсортированная по убыванию вместимости;
- страница локации — адрес, вместимость, проверка занятости на выбранную дату и съёмки на этой локации;
- график съёмок — таблица по датам со статусами (запланирована, прошла, отменена);
- страница съёмки — дата, фильм, сцена, локация (со ссылками) и статус;
- собственная страница ошибки 404.

45. Какие URL соответствуют этим страницам?

Страница	**URL**	**Имя маршрута (name=)**
Главная	`/`	`index`
Список фильмов	`/films/`	`films`
Фильм	`/films/<int:film_id>/`	`film_detail`
Список локаций	`/locations/`	`locations`
Локация	`/locations/<int:location_id>/` (дата: `?date=ГГГГ-ММ-ДД`)	`location_detail`
График съёмок	`/shootings/`	`shootings`
Съёмка	`/shootings/<int:shooting_id>/`	`shooting_detail`
Ошибка 404	любой неизвестный адрес	`handler404`

Маршрутизация двухуровневая: корневой `kinoplan/urls.py` подключает через `include()` файлы `urls.py` приложений `homepage`, `films`, `locations`, `shootings`.

46. Какие view-функции реализованы?

Файл	Функции
`homepage/views.py`	`index(request)`; `page_not_found(request, exception)` — обработчик 404; вспомогательные `page(title, content)` — общий HTML-каркас с Bootstrap и навигацией, `not_found_page()` — страница «объект не найден»
`films/views.py`	`films(request)`, `film_detail(request, film_id)`
`locations/views.py`	`locations(request)`, `location_detail(request, location_id)`
`shootings/views.py`	`shootings(request)`, `shooting_detail(request, shooting_id)`; вспомогательная `status_badge(shooting)` — цветной бейдж статуса

Каждая view-функция принимает `request` и возвращает `HttpResponse`. Функции деталей возвращают код 404, если объекта с таким id нет.

47. Как запрос пользователя проходит от URL до HTML-страницы?

Пример: пользователь открывает `/locations/1/?date=2026-10-20`.

1. Браузер отправляет HTTP-запрос `GET /locations/1/?date=2026-10-20`.
2. Django просматривает `kinoplan/urls.py` сверху вниз. Подходит `path("locations/", include("locations.urls"))`: префикс `locations/` отрезается, остаток `1/` передаётся в `locations/urls.py`.
3. Там совпадает `path("<int:location_id>/", views.location_detail)`. Конвертер `int` превращает «1» в число, и Django вызывает `location_detail(request, location_id=1)`. Строку `?date=...` функция читает из `request.GET`.
4. Функция загружает данные через `storage.py`: `load_locations()`, `load_films()` и `load_shootings()`. JSON-файлы превращаются в объекты `Location`, `Film`, `Shooting`.
5. `find_location_by_id(locations, 1)` ищет локацию. Если результат `None`, возвращается ответ с кодом 404.
6. Дата разбирается через `date.fromisoformat()`. При некорректной дате берётся текущая и выводится предупреждение. Затем `is_location_available()` проверяет, нет ли на этой локации активной съёмки в эту дату.
7. Из атрибутов объекта (`location.name`, `.address`, `.capacity`) собирается HTML-фрагмент `content`. Функция `page()` вставляет его в общий каркас (head, Bootstrap, навигация).
8. View возвращает `HttpResponse(html)` со статусом 200. Django отправляет ответ браузеру, браузер отображает страницу: «20.10.2026: Локация занята на выбранную дату».

Шаблоны (`templates/`, `render()`) на ПР5 ещё не используются: HTML формируется в Python-строке функцией `page()`.

48. Какие элементы проекта перенесены из консольного приложения ПР3?

- **Классы предметной области** из пакета `models/`: `Location`, `Film`, `Scene`, `Shooting`. Страницы выводят атрибуты объектов, а у съёмки используют связи `shooting.film`, `shooting.location`.
- **Хранилище** `storage.py`: функции `load_locations()`, `load_films()` и `load_shootings()` с восстановлением связей по id.
- **Функции обработки**: `find_location_by_id()`, `find_film_by_id()`, `find_shooting_by_id()`, `is_location_available()`, `sort_locations()` (сортировка через lambda), `sort_shootings()`, `check_location_availability()` (функция ещё из ПР1).
- **Методы и свойства**: `__str__()` объектов, свойство `Shooting.status`, метод `days_until()`.

Раньше результат выводился через `print()` в консоль, теперь он показывается на веб-странице. Логика не дублируется: view-функции вызывают готовый код ПР3. Консольная версия `main.py` тоже сохранена и работает. Функции, которые изменяют данные (`add_location()`, `add_film()`, `create_shooting()`, `cancel_shooting()`), в веб-интерфейсе пока не используются: для них нужны формы и POST-запросы.

49. Почему на ПР5 ещё не выполняется полный переход на Django ORM?

Работа идёт поэтапно. На ПР5 изучается путь запроса: HTTP, маршрутизация, view-функции, формирование HTML-ответа. Модели и база данных — следующая большая тема (миграции, ORM, связи моделей). Если менять всё сразу, при ошибке трудно понять, где она: в маршрутах, во view или в базе. Поэтому данные пока берутся из уже проверенной объектной модели ПР3 и JSON-файлов, а веб-слой построен поверх неё. Потом источник данных можно заменить на базу, не переписывая маршруты и страницы с нуля. Методичка прямо указывает, что перенос в ORM на ПР5 не требуется.

50. Как будет изменяться проект после подключения базы данных?

- **Модели.** В `models.py` приложений появятся Django-модели:
  - `Location`: `name`, `address`, `capacity`;
  - `Film`: `title`, `director`;
  - `Scene`: `ForeignKey` на `Film`, `number`, `description`, `crew_size`;
  - `Shooting`: `ForeignKey` на `Film`, `Scene` и `Location`, `shooting_date`, `is_cancelled`.

  Связи, которые сейчас восстанавливаются по id из JSON, будут храниться как внешние ключи.
- **Миграции.** `python manage.py makemigrations` и `migrate` создадут таблицы в базе данных. Данные из `data/*.json` нужно будет один раз перенести в базу.
- **View-функции.** Вместо `load_*()` и `find_*_by_id()` будут запросы ORM: `Location.objects.all()`, `get_object_or_404(Location, id=location_id)`. Проверка занятости станет запросом `Shooting.objects.filter(location=..., shooting_date=..., is_cancelled=False).exists()`.
- **Хранение.** `storage.py` и JSON-файлы перестанут быть основным хранилищем. Данные будут в базе, а ограничения (например, на вместимость) можно будет задать на уровне модели.
- **Админ-зона.** Модели регистрируются в `admin.py`, и фильмы, локации и съёмки можно будет добавлять и изменять через админку.
- **Шаблоны.** Функцию `page()` заменит `base.html` с наследованием шаблонов и `render()`.

51. Какие данные пока используются для отображения страниц?

Данные из JSON-файлов проекта, а не из кода и не из базы данных:

- `data/films.json` — фильмы со сценами внутри;
- `data/locations.json` — локации;
- `data/shootings.json` — съёмки. Связи хранятся как `film_id`, `scene_number`, `location_id`, плюс признак `is_cancelled`.

При каждом запросе view-функция читает файлы через `storage.py` и получает объекты классов ПР3. Веб-интерфейс данные только читает. Изменять их пока можно в консольной версии `main.py`: она сохраняет изменения в те же файлы, и после обновления страницы они видны в браузере. База данных `db.sqlite3` пока содержит только служебные таблицы Django (появились после `migrate`).

52. Какие возможности проекта будут реализованы на следующих практических работах?

- шаблоны Django (`templates/`, `base.html`, наследование шаблонов, контекст) и статические файлы — ПР6;
- модели Location, Film, Scene, Shooting в Django ORM, база данных и миграции — ПР7;
- запросы ORM, связи моделей, админ-зона — ПР9;
- формы (Django Forms, POST): добавление фильмов, сцен, локаций, назначение и отмена съёмок из браузера, то есть полный CRUD; проверки вместимости и занятости будут работать уже в формах — ПР10;
- регистрация и вход пользователей, разграничение доступа (например, назначать съёмки может только ассистент режиссёра или администратор) — ПР11;
- расширенное тестирование pytest и тесты Django — ПР13–ПР15;
- далее по плану проекта: REST API, контейнеризация (Docker), CI/CD.